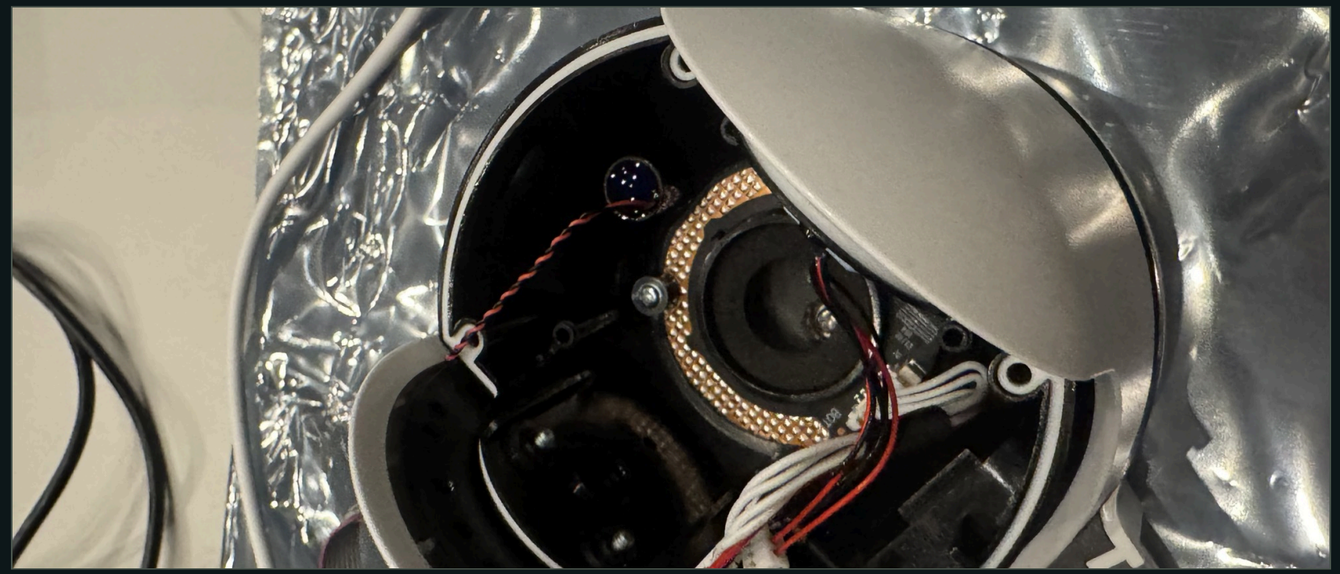
LE DÉFI · DU MATÉRIEL BIEN RÉEL

Elle surveillait le jardin.
On a regardé à l’intérieur.



TAPO C500 2.0

Une caméra.
Un petit ordinateur.

La mémoire conserve le logiciel,
les réglages et des fichiers.

NOTRE OBJECTIF

BA26{…}

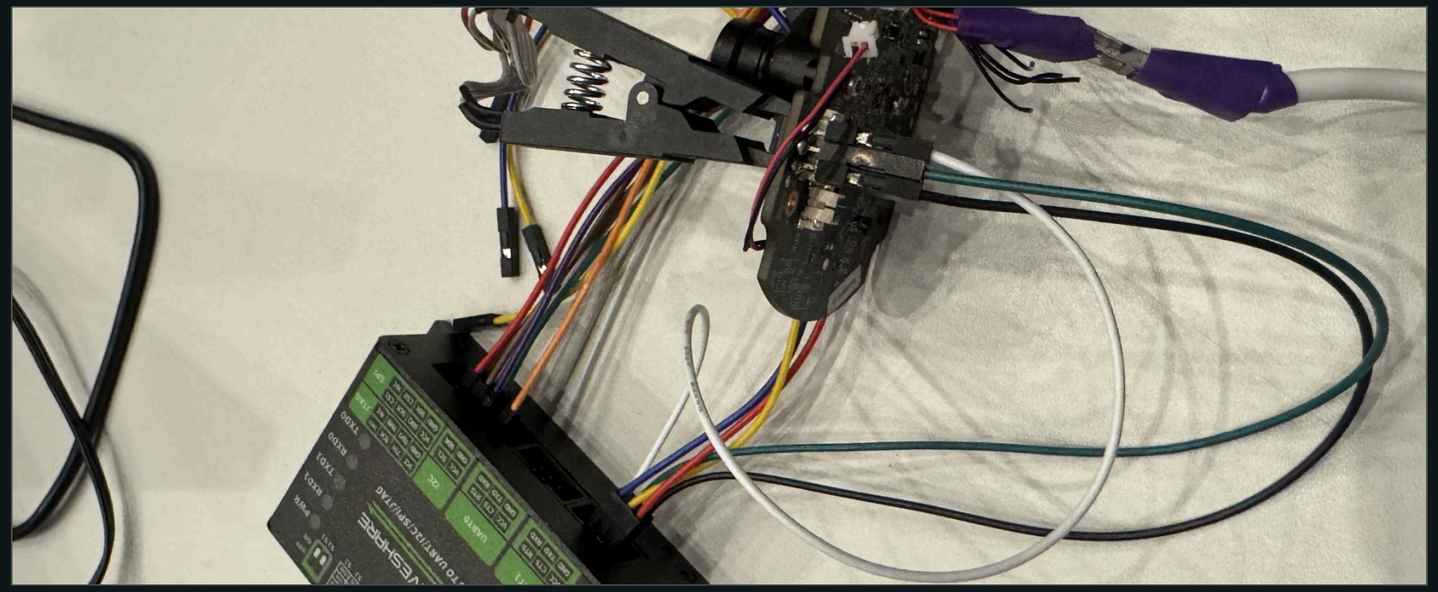
Retrouver le message caché.

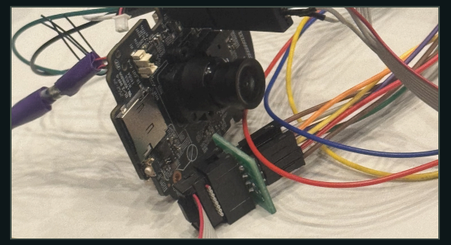
L’IDÉE À RETENIR

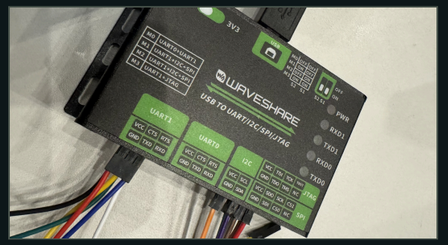
Le point de départ de l’analyse : **une copie de la mémoire de 8 Mio.**

L’ACQUISITION · COPIER LA MÉMOIRE POUR L’ÉTUDIER

Beaucoup de fils.
À l’arrivée : un fichier.







Mémoire flash

↓ lecture

Adaptateur → ordinateur

↓ copie

dump · 8 Mio

L’IDÉE À RETENIR

On poursuit l’enquête **dans le fichier récupéré.**

PREMIER ESSAI · LA RECHERCHE DE TEXTE NE SUFFIT PAS

Ctrl+F n’a rien trouvé.
Il fallait changer d’échelle.

01 / STOCKAGE

Des fichiers compressés

Prendre moins de place.



→ `fichiers`

Décompresser pour accéder au contenu.

02 / MESSAGE

Un flag chiffré

Rendre le texte illisible.

`9c b6 c8 70 d8 3a…`

Comprendre la recette pour le retrouver.

L’IDÉE À RETENIR

Ce défi combine **compression des fichiers** et **chiffrement du message.**

LE DÉCLIC · UNE ZONE UTILE AVAIT ÉCHAPPÉ À L'EXTRACTION

Le livre avait perdu sa couverture. Pas ses pages.

DUMP · SCHÉMA NON À L'ÉCHELLE



DÉBUT NON RECONNAISSABLE

?

La première extraction ne voit pas tout.

→

MÉTADONNÉES INTACTES

Position	Où lire
Taille	Combien lire
Fragment	Quel bloc ouvrir

→

FICHIERS RÉCUPÉRÉS

`ctf_unlock`

`flag.enc`

Lire les bons octets.

L'IDÉE À RETENIR

L'outil n'avait pas tout vu. La mémoire avait encore des choses à raconter.

LES INDICES · TROIS PIÈCES QUI SE COMPLÈTENT

Le message était secret.
Le mode d’emploi était à côté.

01

flag.enc

Le message chiffré.
35 octets dans le fichier fourni.

+

02

ctf_unlock

Le programme qui explique comment retrouver le texte.

+

03

devId

L’identifiant de la caméra, présent dans ses réglages.

AU DÉMARRAGE

```
devId → clé temporaire → ctf_unlock + flag.enc → /tmp/flag.txt
```

L’IDÉE À RETENIR

Le démarrage relie les indices : **identifiant + programme + message chiffré.**

LE MÉCANISME · REPRODUIRE LA RECETTE

Mêmes ingrédients.
Exactement le même masque.

`devId · identifiant` + `« C500-CTF-v1 »` + `compteur : 0, puis 1`

↓ `SHA-256` Une empreinte de 32 octets par compteur.

`Bloc 0 · 32 octets` `Bloc 1 · 32 octets`

Le message fait **35 octets** : on utilise le premier bloc, puis 3 octets du second.

L'IDÉE À RETENIR

On calcule SHA-256 dans le sens habituel. **On ne l'inverse pas.**

PETITE PAUSE · LE PUBLIC PEUT AUSSI DÉCHIFFRER

Une opération qui sait faire marche arrière.

XOR

Deux bits identiques → **0**

Deux bits différents → **1**

Exemple pédagogique de 4 bits.
Ce ne sont pas les données du flag.

Message

1 0 1 0

Masque

0 1 1 0

Chiffré

1 1 0 0

Chiffré

1 1 0 0

Même masque

0 1 1 0

Message

1 0 1 0

L'IDÉE À RETENIR

Appliquer **le même masque deux fois** rend le message d'origine.

DU SCHÉMA AU CODE · UN EXTRAIT, TROIS GESTES

La recette tient en Python. Chaque ligne a un rôle.

01	`key + b'C500-CTF-v1' + struct.pack('<I', counter)`	Assembler les ingrédients. Le compteur prend 0, puis 1.
02	`hashlib.sha256(…).digest()`	Produire un bloc de 32 octets. Mettre les deux blocs bout à bout.
03	`bytes(a ^ b for a, b in zip(encrypted, stream))`	Combiner chaque octet chiffré avec l'octet du masque.

Extraits de solve.py, séparés pour la lecture ; « … » abrège les ingrédients. Le script n'exécute pas ctf_unlock.

L'IDÉE À RETENIR

Le script relie les fichiers extraits, l'identifiant et la recette. **Le résultat devient vérifiable.**

LE RÉSULTAT · TEXTE CONSERVÉ DANS FLAG.TXT

Souriez.
Le flag est sorti.

CONTENU EXACT DU FICHIER FLAG.TXT FOURNI

```
BA26{a-R00t-0f-Tru5t-Sh1ps-N0-K3ys}
```

35 caractères · même résultat que dans le PowerPoint.

ÉTAT DOCUMENTÉ **Flag validé.** Soumission confirmée par la plateforme.

L'IDÉE À RETENIR

Le secret partageait sa mémoire avec **sa recette et les données nécessaires pour le lire.**